

Laboratorio 4 — Fundamentos de Aprendizaje por Refuerzo y Gymnasium

CC3092 · Deep Learning y Sistemas Inteligentes

1. Fundamentos del aprendizaje por refuerzo

El aprendizaje por refuerzo (RL) estudia cómo un *agente* aprende a decidir interactuando con un *entorno* para maximizar la recompensa acumulada en el tiempo. A diferencia del aprendizaje supervisado (aprende de pares entrada–etiqueta, con señal instructiva) y del no supervisado (busca estructura en datos sin etiqueta), su señal es *evaluativa* —indica qué tan buena fue una acción, no cuál era la correcta—, las acciones afectan los estados y las recompensas futuras, y la recompensa puede llegar con retraso (problema de asignación de crédito).

Sus componentes son el agente (decide y aprende), el entorno (todo lo externo), el estado s , la acción a , la recompensa r (que define el objetivo) y la política $\pi(a|s)$ (la estrategia que mapea estados a acciones). El marco formal es el Proceso de Decisión de Markov (MDP), la tupla (S, A, P, R, γ) : estados, acciones, función de transición $P(s'|s, a)$, función de recompensa R y factor de descuento $\gamma \in [0, 1]$ que pondera el futuro; el objetivo es maximizar el retorno $G_t = \sum \gamma^k r_{t+k+1}$.

Las funciones de valor miden calidad esperada: $V(s)$ es el retorno esperado desde s siguiendo π , y $Q(s, a)$ el retorno tras tomar a en s y luego seguir π ; la política óptima es *greedy* respecto de Q^* , es decir $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$. La ecuación de Bellman descompone el valor de forma recursiva (recompensa inmediata más el valor descontado del estado siguiente) y es la base para calcular las funciones de valor.

El dilema exploración vs. explotación equilibra aprovechar la mejor acción conocida y probar otras para descubrir mejores opciones; dos estrategias son ϵ -*greedy* (acción aleatoria con probabilidad ϵ) y *softmax/Boltzmann* (probabilidad $\propto \exp(Q/\tau)$). Se distinguen además tareas episódicas y continuas, métodos on-policy (mejoran la misma política que actúa) y off-policy (aprenden otra distinta), y model-based (usan un modelo de P y R) vs. model-free. Q-Learning (model-free, off-policy) actualiza $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, donde α (tasa de aprendizaje) regula cuánto se corrige cada estimación y γ cuánto pesan las recompensas futuras. (Fuentes: Sutton & Barto, 2018; Watkins & Dayan, 1992.)

2. La librería Gymnasium

Gymnasium (Farama Foundation) provee una API estándar para entornos de RL y una colección de entornos de referencia; es el *fork* mantenido y sucesor de OpenAI Gym, y desacopla el agente del entorno, lo que favorece la reproducibilidad y el *benchmarking*. Un entorno se crea con `gym.make(id)` y expone `reset(seed)` $\rightarrow$ (observation, info), que inicia un episodio; `step(action)` $\rightarrow$ (observation, reward, terminated, truncated, info), donde *terminated* marca un estado terminal del MDP y *truncated* un corte externo (p. ej. el límite de pasos); además `render()` para visualizar y `close()` para liberar recursos.

Los espacios definen las observaciones y acciones válidas: *Discrete*(n) (conjunto finito), *Box*(*low*, *high*, *shape*) (región continua multidimensional) y *MultiDiscrete* (vector de enteros con rangos propios). El catálogo agrupa familias como Classic Control, Toy Text y Box2D; por ejemplo CartPole-v1 (obs Box(4), acc. Discrete(2)), FrozenLake-v1 (obs Discrete(16), acc. Discrete(4)), MountainCar-v0 (obs Box(2), acc. Discrete(3)) y LunarLander-v3 (obs Box(8), acc. Discrete(4)). Un *wrapper* envuelve un entorno para modificar su comportamiento sin tocar el base (TimeLimit, RecordVideo, normalización de observaciones) y son componibles. (Fuente: documentación de Gymnasium.)

3. Módulo de prueba: resultados

Se ejecutó un agente aleatorio (`action_space.sample()`) durante 20 episodios en CartPole-v1 y FrozenLake-v1, y una política simple *no aprendida* en CartPole-v1 que empuja hacia el lado al que cae el

poste (según la velocidad angular, obs[3]). Con semillas fijas, el retorno es la recompensa total por episodio (en CartPole, +1 por paso $\Rightarrow$ retorno = pasos sobrevividos).

Entorno	Agente	Retorno medio	Desv. estándar	Nota
CartPole-v1	aleatorio	19.5	9.6	recompensa densa (+1/paso)
CartPole-v1	política simple	194.8	34.4	$\approx 10\times$ mejor, sin aprender
FrozenLake-v1	aleatorio	0.05	0.22	1/20 éxitos ($\approx 2.4\%$ en 500 ep.)

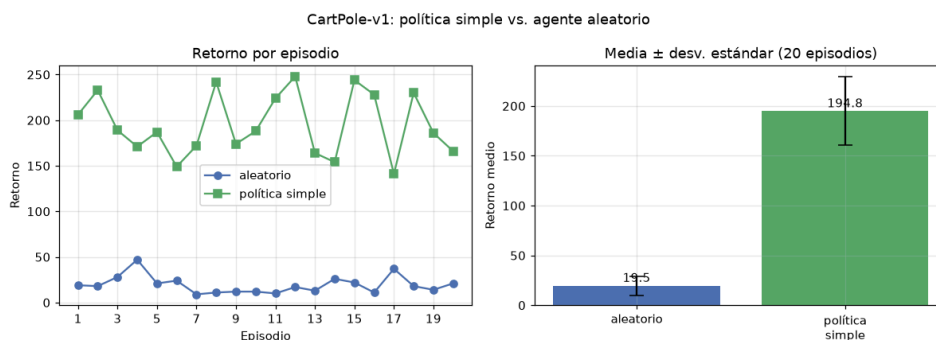


Fig. 1. CartPole-v1: retorno por episodio (izq.) y media $\pm$ desv. estándar (der.) de la política simple frente al agente aleatorio.



Fig. 2. Agente aleatorio en FrozenLake-v1: recompensa por episodio (1 = llegó a la meta).

4. Discusión y análisis

Diferencias del agente aleatorio entre entornos. En CartPole el agente aleatorio sobrevive ≈ 20 pasos porque la recompensa es densa (+1 por paso) y cualquier secuencia acumula recompensa mientras el poste no caiga; en FrozenLake apenas resuelve el entorno (1/20; $\approx 2.4\%$ en 500 episodios) porque la recompensa es dispersa (1 solo al llegar a la meta), la superficie es resbaladiza y solo trayectorias concretas tienen éxito. La diferencia refleja la naturaleza de cada entorno: recompensa densa frente a dispersa.

Política simple vs. aleatoria. La política simple superó ampliamente a la aleatoria (194.8 vs. 19.5 de retorno medio, $\approx 10\times$) sin aprender nada. Esto muestra que incorporar conocimiento del dominio da un punto de partida muy superior a empezar desde cero; aun así, una política aprendida puede superar a la regla fija y adaptarse a situaciones que esta no cubre.

Por qué explorar aun con una política razonable. Una política solo puede ser óptima respecto de lo que ya conoce; sin exploración no descubre acciones o estados potencialmente mejores y queda atrapada en un óptimo local. Explotar maximiza la recompensa inmediata, pero explorar es lo que permite mejorar la estimación de Q y hallar la política óptima: ese es el compromiso exploración vs. explotación.

Entorno más interesante para un laboratorio futuro. FrozenLake-v1: al tener observación y acción discretas y pequeñas (Discrete(16) y Discrete(4)), permite representar Q como una tabla exacta 16×4 y aplicar directamente la actualización de Q -Learning sin aproximación de funciones, mientras que su estocasticidad y recompensa dispersa lo mantienen como un reto real. CartPole o LunarLander, con observaciones continuas, exigirían aproximar Q (p. ej. DQN), un paso más complejo para un primer agente aprendido.